

Ficha técnica — CertManager

Campo	Valor
Nombre del aplicativo	CertManager
Versión de entrega	1.0.0
Descripción	Monitoreo de certificados SSL/TLS multi-grupo con alertas, reportes y API REST.
Tipo	Aplicación web (server-rendered) + API REST
Criticidad	Media (operativo / seguridad; no transaccional, sin datos de pago)
Propietario funcional (Owner)	Gerencia de Soporte Producción Sistemas Serv. al Cte.
Lenguaje / framework	Python 3.11+ (probado en 3.13) · Django 5.2 · Django REST Framework
UI	Server-rendered (design system Forge UI) + Tailwind + HTMX
Servidor de aplicación	Gunicorn (WSGI) detrás de NGINX
Base de datos	MySQL 8 (producción) · SQLite (pruebas)
Tareas programadas	APScheduler en-proceso (<code>manage.py run_scheduler</code>)
Autenticación	Local (email+contraseña) y/o LDAP/Active Directory · API por API key
Roles	Owner global + por grupo: VIEWER / CONTRIBUTOR / ADMIN

Funcionalidades principales

- **Monitoreo:** conexión TLS periódica a cada host, parseo del certificado, días restantes, estado.
- **Alertas:** plataforma (in-app), correo (SMTP), webhook (Teams/Slack), SMS (gateway FTP).
- **Reportes** programados (PDF / Excel / correo).
- **Gestión** de certificados, grupos, usuarios, plantillas de correo.
- **API REST** (certificados, grupos, alertas) autenticada por API key.

Interfaces externas

Interfaz	Protocolo / Puerto	Sentido
Usuarios / API	HTTPS 443	Entrante
Base de datos MySQL	TCP 3306	Saliente
Hosts monitoreados	TLS (443 u otro por cert)	Saliente
SMTP	587/465/25	Saliente

Interfaz	Protocolo / Puerto	Sentido
LDAP / AD	389 / 636	Saliente
Gateway SMS	FTP 21	Saliente
Webhooks	HTTPS 443	Saliente

Datos que gestiona

Certificados (dominios, umbrales, estado), destinatarios (correos), grupos, usuarios, alertas, reportes, plantillas de correo, configuración, registro de auditoría. **No** procesa datos de tarjeta ni pagos.

Librerías utilizadas y versiones

Dependencias de la entrega 1.0.0. Las versiones corresponden a las instaladas en el entorno entregado; las exclusivas de producción / opcionales se indican con su restricción de `requirements/` (no van en el instalable mínimo local).

Librería	Versión	Uso
Django	5.2.15	Framework web (ORM, vistas, admin).
django-rest-framework	3.17.1	API REST.
django-filter	25.2	Filtros de la API.
django-environ	0.13.0	Configuración por entorno / <code>.env</code> .
drf-spectacular	0.29.0	Documentación OpenAPI/Swagger de la API.
django-csp	4.0	Cabeceras Content-Security-Policy.
whitenoise	6.12.0	Servir estáticos (precompresión brotli/gzip, off-CDN).
cryptography	49.0.0	Parseo de certificados (SAN, cadena, fingerprint).
APScheduler	3.11.2	Planificador en-proceso (monitoreo + reportes).
python-dateutil	2.9.0	Recurrencia de reportes programados (rrule).
requests	2.34.2	Envío de webhooks (Teams/Slack/genérico).
pyotp	2.9.0	2FA TOTP (RFC 6238).
qrcode	8.2	QR de enrolamiento 2FA.
Pillow	12.2.0	Avatares y logo de organización.
ldap3	2.9.1	Autenticación LDAP / Active Directory.
reportlab	4.5.1	Exportación de reportes a PDF.
openpyxl	3.1.5	Exportación de reportes a Excel (.xlsx).
gunicorn	≥ 21.2 (producción)	Servidor WSGI.
mysqlclient	≥ 2.2 (producción)	Driver MySQL 8.

Librería	Versión	Uso
weasyprint	≥ 62.0 (producción)	PDF de reportes con HTML/CSS.
obsforge[django]	0.1.3 (opcional, índice privado)	Observabilidad/logging estructurado.

Detalle completo y restricciones en `requirements/base.txt` y `requirements/prod.txt`. CI ejecuta `pip-audit` (**0 vulnerabilidades** conocidas).

Dimensionamiento mínimo

2 vCPU / 2 GB RAM / 10 GB disco (recomendado 4 vCPU / 4 GB). Ver doc 02 §2.

Estado de calidad

- **529 pruebas automatizadas** (verde).
- CI: chequeo de sistema + migraciones + suite + auditoría de CVEs (`pip-audit`).
- `pip-audit` : **0 vulnerabilidades** conocidas en dependencias.
- Despliegue verificado end-to-end en Docker, Linux y Windows.